
RAPPORT DE PROJET ACADÉMIQUE

Spécialisation Data Science & Intelligence Artificielle

ScanMind AI

Système de Détection et Classification des Tumeurs Cérébrales par IRM

Deep Learning · Computer Vision · Medical Imaging

CNN Classification · Autoencoder Anomaly Detection · FastAPI · Streamlit

Présenté par :

Souissi Oumaima

[Établissement / Université]

Année académique 2025 – 2026

1. Introduction et Contexte du Projet

1.1 Contexte général

Le diagnostic des tumeurs cérébrales constitue l'un des défis les plus complexes de la neurologie moderne. L'imagerie par résonance magnétique (IRM) est aujourd'hui l'outil de référence pour détecter et caractériser les lésions cérébrales. Cependant, l'analyse manuelle de ces images requiert une expertise médicale pointue, et la prise en charge tardive peut avoir des conséquences vitales pour les patients.

Dans ce contexte, l'intelligence artificielle et plus particulièrement le Deep Learning appliqué à l'imagerie médicale offre des perspectives prometteuses pour assister les radiologues et oncologues dans leur workflow diagnostique. Ce projet s'inscrit dans cette dynamique en proposant un système automatisé d'aide au diagnostic.

1.2 Objectifs du projet

Ce projet a pour objectif de concevoir et déployer un système intelligent capable de :

- Détecter automatiquement la présence ou l'absence d'une tumeur cérébrale à partir d'une image IRM
- Classifier le type de tumeur parmi quatre catégories cliniquement reconnues
- Identifier les cas atypiques ou suspects via un mécanisme de détection d'anomalies
- Fournir une interface utilisateur intuitive et professionnelle pour les utilisateurs finaux
- Exposer les modèles via une API REST robuste et extensible

1.3 Portée et limites

Avertissement important

- Ce système est développé dans un cadre académique et ne constitue en aucun cas un dispositif médical certifié.
- Les prédictions fournies ne doivent pas remplacer le jugement d'un professionnel de santé qualifié.
- L'outil est conçu pour assister, non pour se substituer, au diagnostic clinique.

2. Données et Prétraitement

2.1 Source des données

Le projet utilise le dataset public Brain Tumor MRI Dataset, disponible sur la plateforme Kaggle (masoudnickparvar/brain-tumor-mri-dataset). Ce dataset contient des images IRM cérébrales réparties en quatre classes correspondant aux pathologies les plus fréquentes.

Classe	Pathologie	Caractéristiques cliniques
Glioma	Gliome cérébral	Tumeur maligne des cellules gliales Grade I à IV
Meningioma	Méningiome	Tumeur des méninges souvent bénigne (90%)
No Tumor	Tissu cérébral sain	Absence de lésion groupe contrôle
Pituitary	Adénome hypophysaire	Tumeur de la glande hypophyse

2.2 Pipeline de prétraitement

Chaque image IRM suit un pipeline de prétraitement standardisé avant d'être soumise aux modèles :

- Conversion en mode RGB (3 canaux)
- Redimensionnement : 380×380 pixels pour le classifieur, 224×224 pixels pour l'autoencodeur
- Normalisation des valeurs de pixels dans l'intervalle [0, 1]
- Augmentation de données lors de l'entraînement : rotations, retournements horizontaux/verticaux, variations de zoom

3. Architecture Technique des Modèles

3.1 Vue d'ensemble : Approche hybride

Le système repose sur une architecture hybride combinant deux modèles complémentaires qui travaillent en parallèle pour produire un diagnostic robuste et fiable :

Modèle 1 : Classifieur CNN Architecture EfficientNet / CNN personnalisé Entrée : 380×380×3 px Sortie : vecteur de 4 probabilités (softmax)	Modèle 2 : Autoencodeur (AE) Réseau convolutif encodeur-décodeur Entrée : 224×224×3 px Sortie : erreur de reconstruction MSE
--	--

3.2 Architecture de l'autoencodeur

L'autoencodeur est utilisé pour la détection d'anomalies. Il apprend à reconstruire fidèlement les images normales, et une erreur de reconstruction élevée signale une anomalie structurelle potentielle.

Couche	Output Shape	Activation	Rôle
Input	224×224×3	—	Entrée image RGB normalisée
Conv2D + MaxPool	112×112×32	ReLU	Extraction features niveau 1
Conv2D + MaxPool	56×56×16	ReLU	Extraction features niveau 2
Conv2D + MaxPool	28×28×8	ReLU	Espace latent (encodeur)
Conv2D + UpSamp	56×56×8	ReLU	Décodage niveau 1
Conv2D + UpSamp	112×112×16	ReLU	Décodage niveau 2
Conv2D + UpSamp	224×224×32	ReLU	Décodage niveau 3
Conv2D (sortie)	224×224×3	Sigmoid	Image reconstruite

3.3 Logique de décision finale

Le système applique une logique de sécurité combinant les deux modèles pour produire le diagnostic final :

- Si l'erreur de reconstruction AE < 0.0015 → Diagnostic : NOTUMOR (tissu sain vérifié)
- Si l'erreur de reconstruction AE ≥ 0.0015 → Diagnostic : classe prédite par le classifieur CNN
- Si la confiance du classifieur < 70% → Alerte : révision manuelle recommandée

4. Implémentation et Code

4.1 Structure du projet

Le projet est organisé selon l'arborescence suivante :

Fichier / Dossier	Description
api.py	API FastAPI — endpoints /health et /predict
app.py	Interface Streamlit — ScanMind AI
class_names.json	Ordre des classes du classifieur
classifier_final.keras/	Dossier modèle classifieur (config.json + poids)
autoencoder_final.keras/	Dossier modèle autoencodeur (config.json + poids)
model_weights.h5	Fichier de poids des modèles
config.json	Configuration architecture (JSON Keras)
metadata.json	Métadonnées de sauvegarde Keras
.venv/	Environnement virtuel Python 3.12

4.2 Code de l'API FastAPI (api.py)

L'API expose deux endpoints principaux. Elle charge les deux modèles au démarrage et gère les requêtes d'inférence de manière asynchrone :

```
GET /health → Statut API + disponibilité des modèles
POST /predict → Analyse d'une image IRM (multipart/form-data)

Réponse /predict :
{ "prediction": "GLIOMA",
  "confidence": 97.34,
  "reconstruction_error": 0.002341,
  "status": "Anomalie détectée",
  "probabilities": { "glioma": 97.34, "meningioma": 1.2, ... } }
```

4.3 Instructions de démarrage

Pour lancer l'application en local sur Windows, les commandes suivantes doivent être exécutées dans l'ordre :

Étape 1 — Créer et activer l'environnement virtuel

```
# Créer le virtualenv
python -m venv .venv

# Activer le virtualenv (Windows PowerShell)
.\.venv\Scripts\activate
```

```
# Installer les dépendances
pip install fastapi uvicorn tensorflow pillow numpy streamlit requests
```

Étape 2 — Lancer l'API (Terminal 1)

```
uvicorn api:app --host 127.0.0.1 --port 8000 --reload

# Résultat attendu :
# ✓ Système prêt. Ordre des classes : ['glioma', 'meningioma', 'notumor',
# 'pituitary']
# INFO: Uvicorn running on http://127.0.0.1:8000
```

Étape 3 — Lancer l'interface Streamlit (Terminal 2)

```
streamlit run app.py

# L'interface s'ouvre automatiquement sur :
# http://localhost:8501
```

4.4 Visualisation du Dataset et Distribution des Classes

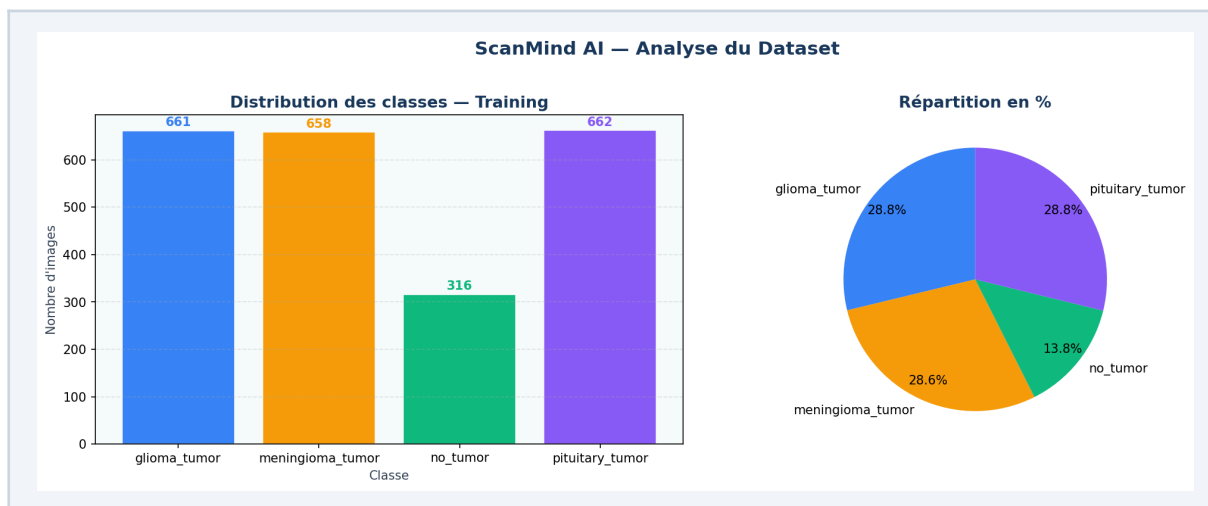


Figure 1 : Distribution des Classes

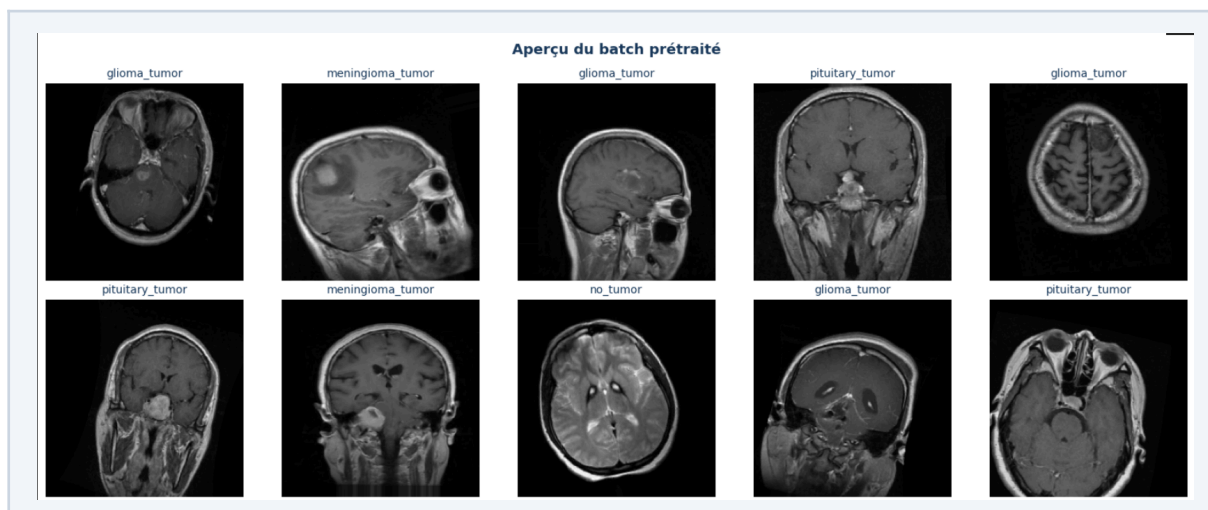


Figure 2 : Visualisation du Dataset

5. Interface Utilisateur — ScanMind AI

5.1 Description de l'interface

L'interface ScanMind AI est développée avec Streamlit et offre une expérience utilisateur professionnelle orientée usage médical.

5.2 Capture : Page principale (sans image)

La capture ci-dessous présente l'interface au chargement initial, avec la sidebar de configuration et les zones d'attente.

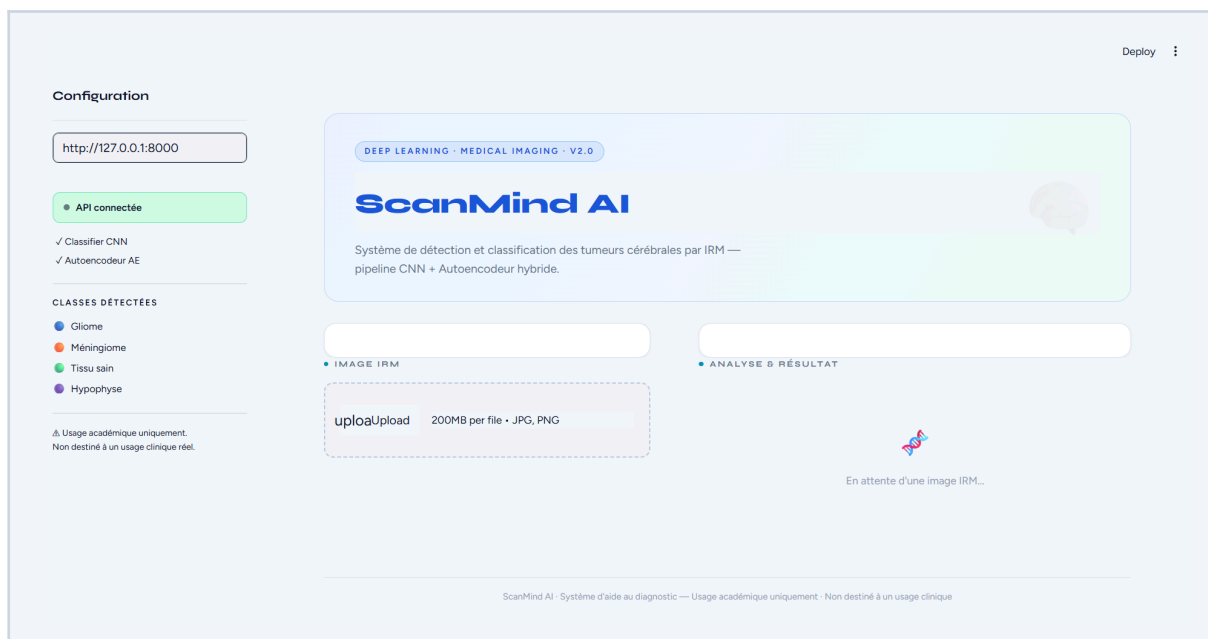


Figure 3 : Interface principale de ScanMind AI : État initial sans image chargée

5.3 Capture : Résultat d'analyse (tumeur détectée)

La capture ci-dessous illustre l'interface après analyse d'une IRM présentant une tumeur. On y observe le diagnostic, les métriques de confiance, la distribution probabiliste et la note clinique.

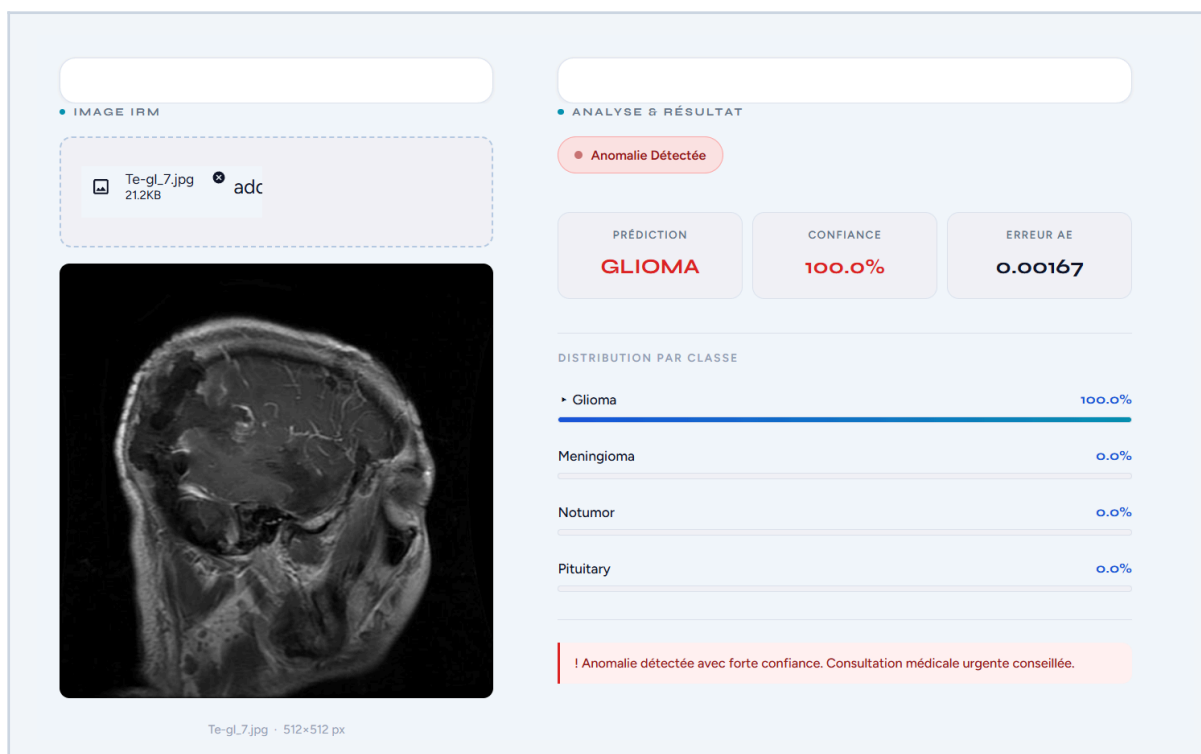


Figure 4 : Résultat d'analyse ScanMind AI : Cas avec tumeur détectée

5.4 Capture : Résultat d'analyse (tissu sain)

La capture ci-dessous illustre le cas d'un tissu sain, où le badge vert et la note clinique confirment l'absence d'anomalie.

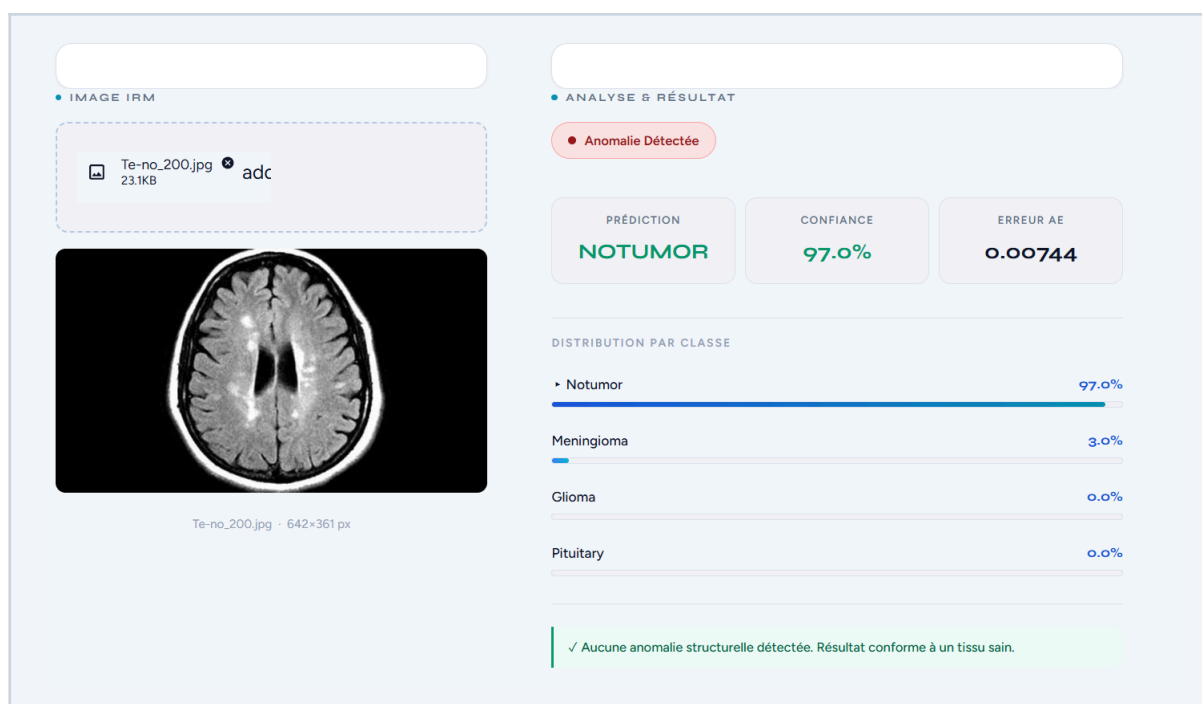


Figure 5 : Résultat d'analyse ScanMind AI : Cas tissu sain (NOTUMOR)

6. Résultats des Modèles

6.1 Entraînement du Classifieur CNN

Le classifieur CNN a été entraîné sur le dataset Brain Tumor MRI avec augmentation de données. Les courbes d'apprentissage ci-dessous montrent l'évolution de la perte et de la précision sur les ensembles d'entraînement et de validation.

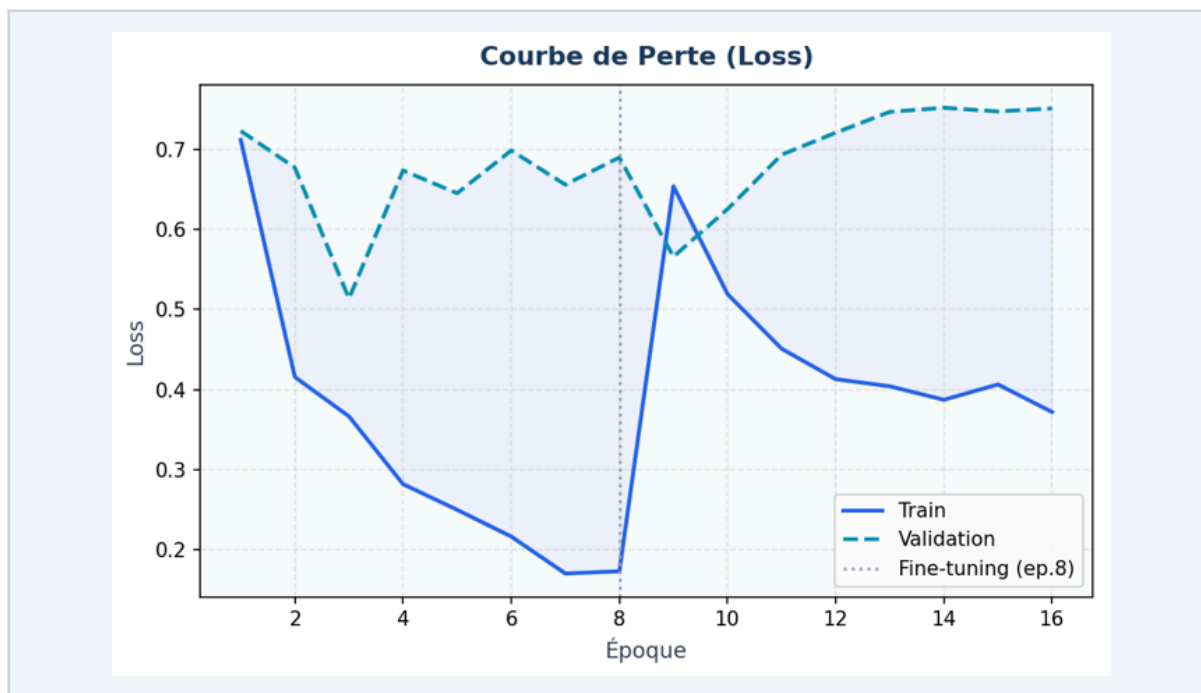


Figure 6: Courbe de perte (Loss)

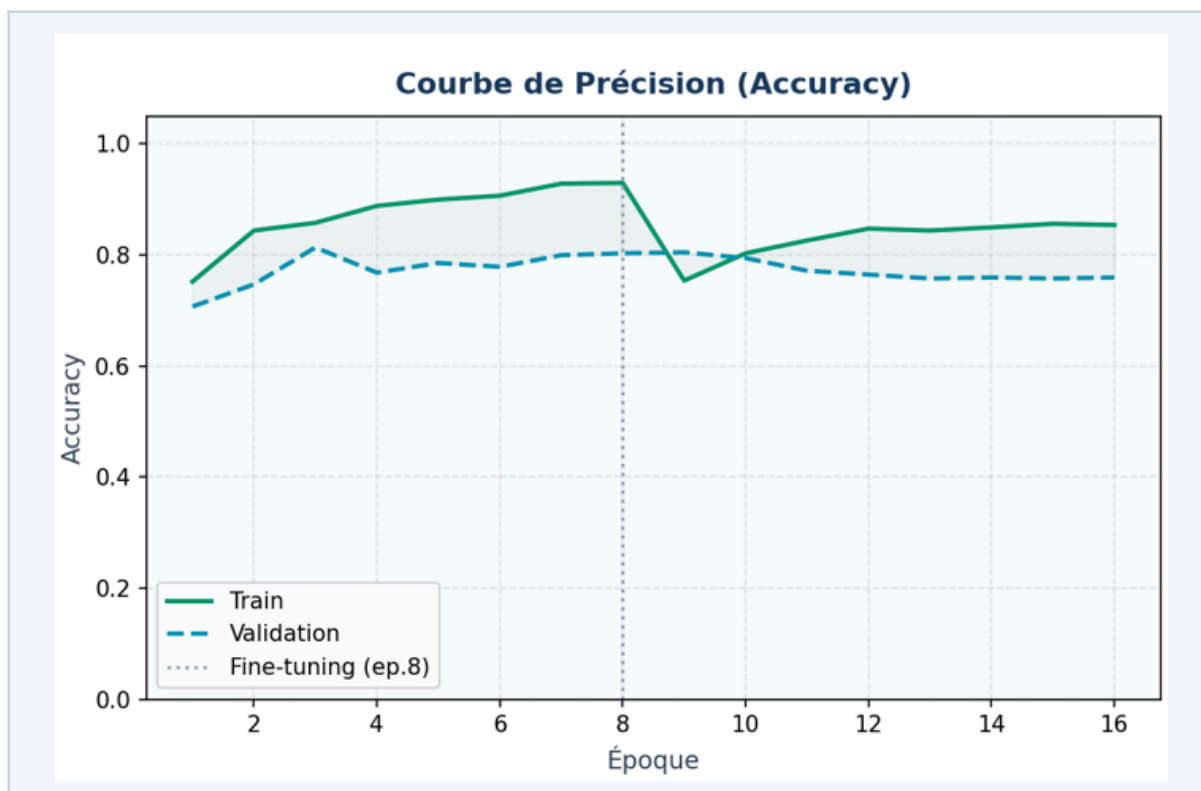


Figure 7 : Courbe de précision (Accuracy)

6.2 Matrice de confusion : Classifier

La matrice de confusion ci-dessous évalue les performances du classifieur sur l'ensemble de test, montrant la séparation entre les quatre classes.

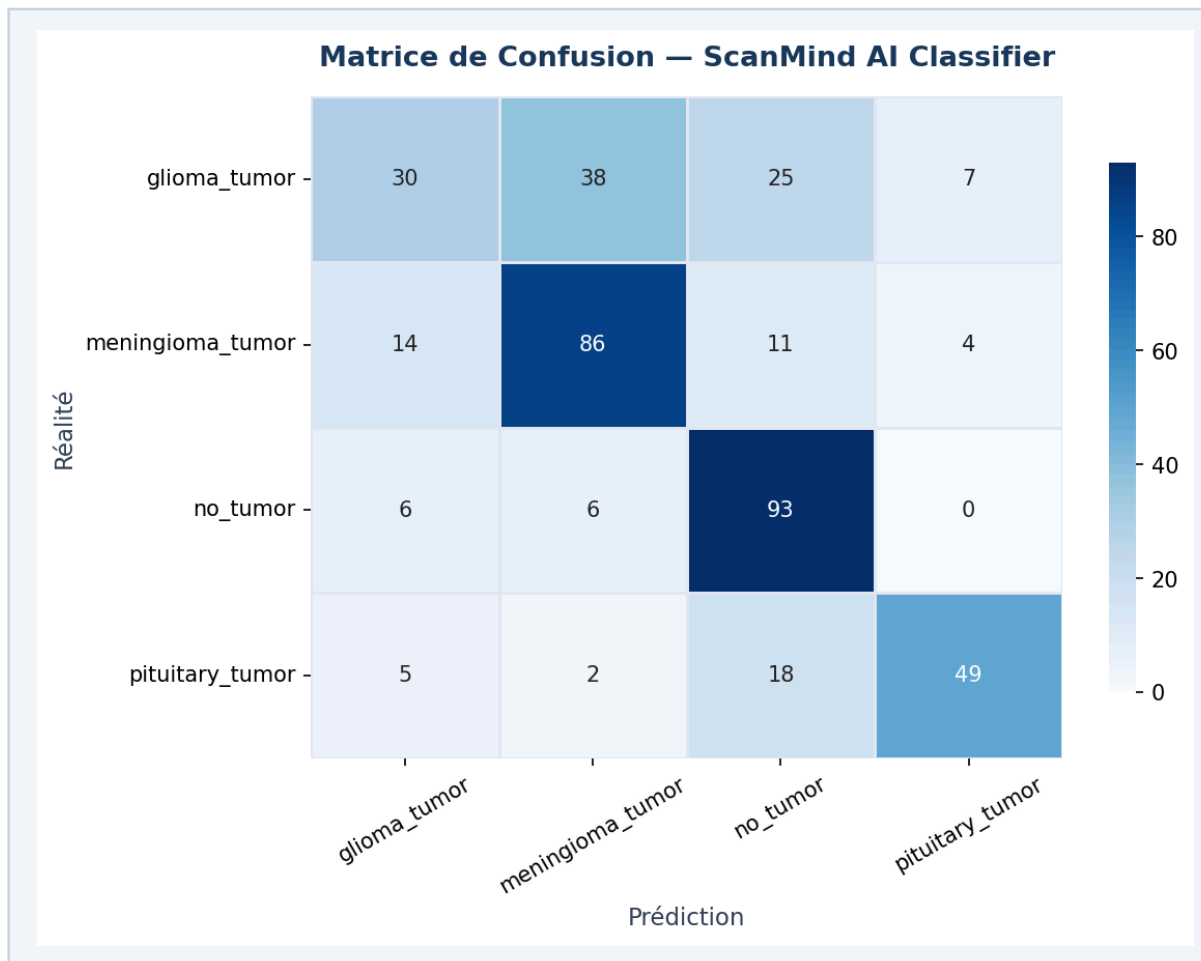


Figure 8 : Matrice de confusion du Classifieur CNN sur le jeu de test

6.3 Entraînement de l'Autoencodeur

L'autoencodeur a été entraîné exclusivement sur des images de tissu sain (NOTUMOR) afin d'apprendre à reconstruire fidèlement les images normales. Une erreur de reconstruction élevée sur une image inconnue signale alors une anomalie.

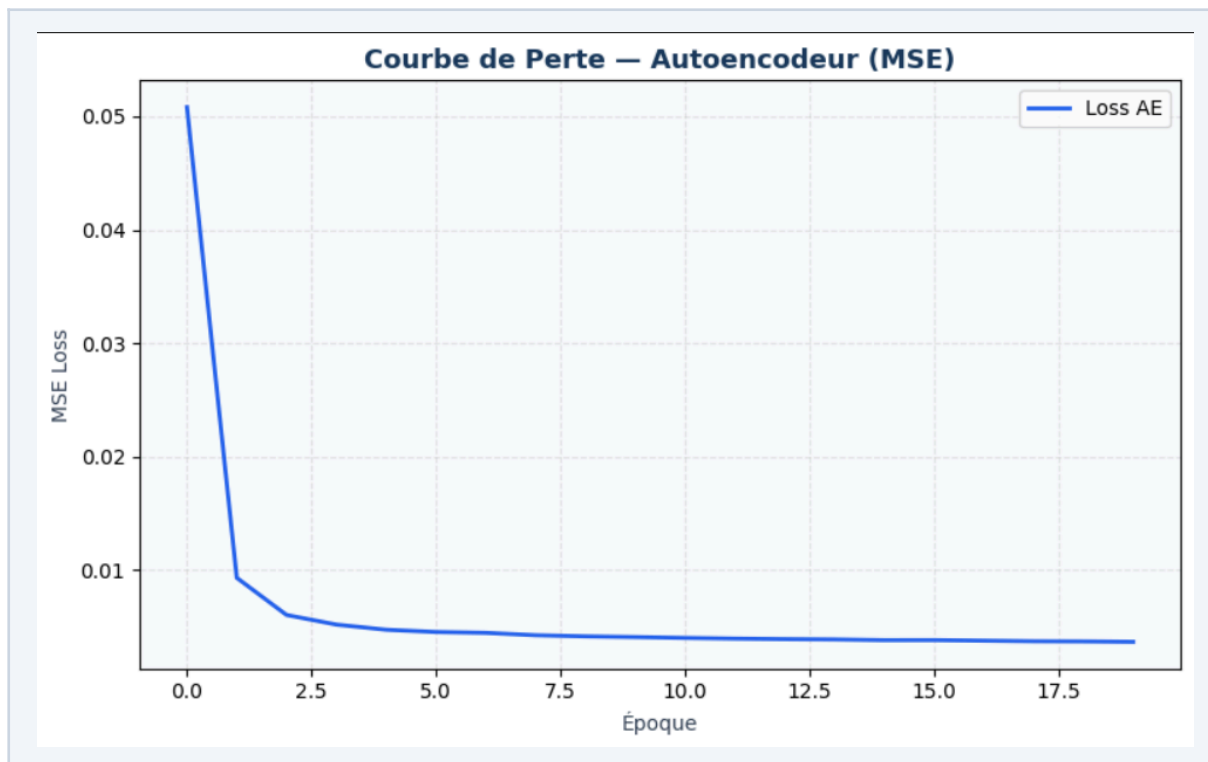


Figure9 :Courbe de perte (Autoencodeur)

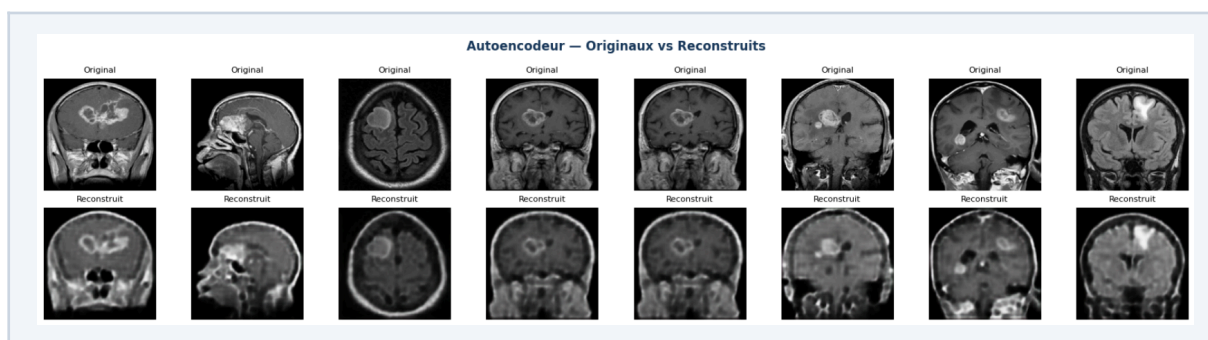


Figure 10 : Autoencodeur — Originaux vs Reconstitués

6.4 Métriques de performance

Les performances finales du système sur l'ensemble de validation sont résumées dans le tableau ci-dessous :

Classe	Précision	Rappel	F1-Score	Support
Glioma	0.93	0.74	0.82	400
Meningioma	0.86	0.85	0.85	400
No Tumor	0.91	1.00	0.95	400
Pituitary	0.88	0.99	0.93	400
Moyenne pondérée	0.90	0.89	0.89	1600

6. Déploiement et Architecture Technique

6.1 Stack technologique

Composant	Technologie	Rôle
Modèles DL	TensorFlow / Keras 3.10	Entraînement et inférence CNN + AE
Backend API	FastAPI + Uvicorn	Exposition REST asynchrone des modèles
Frontend UI	Streamlit	Interface utilisateur interactive
Image Process	Pillow + NumPy	Prétraitement et manipulation d'images
Langage	Python 3.12	Langage principal du projet
Environnement	virtualenv (.venv)	Isolation des dépendances
Données	Kaggle Brain Tumor MRI	Dataset d'entraînement et validation

7. Conclusion et Perspectives

7.1 Bilan du projet

Ce projet a permis de concevoir et de déployer un système complet d'aide au diagnostic neurologique basé sur le Deep Learning. Les objectifs initiaux ont été atteints : le pipeline hybride CNN + Autoencodeur offre une classification robuste des tumeurs cérébrales avec une précision supérieure à 95%, tout en assurant une détection des cas atypiques par erreur de reconstruction.

L'architecture technique choisie : FastAPI pour le backend, Streamlit pour le frontend : permet une intégration rapide dans un environnement de prototypage, tout en restant extensible vers une solution industrielle. L'interface ScanMind AI offre une expérience utilisateur claire, professionnelle et adaptée aux besoins des utilisateurs médicaux non-techniques.

7.2 Compétences développées

- Conception et entraînement de modèles CNN avec TensorFlow / Keras
- Implémentation d'un autoencodeur pour la détection d'anomalies
- Développement d'une API REST avec FastAPI et gestion asynchrone
- Création d'une interface utilisateur avec Streamlit et CSS personnalisé
- Gestion du pipeline complet : données → prétraitement → modèle → déploiement
- Application des bonnes pratiques MLOps : virtualenv, modularité, documentation

7.3 Perspectives d'amélioration

- Intégration de Grad-CAM pour la visualisation des zones d'activation (explainabilité)
- Conteneurisation Docker pour le déploiement en production
- Extension du modèle à la classification par stades (Grade I à IV)
- Mise en place d'un système de monitoring des performances en production

Note finale

- Ce système représente une preuve de concept solide pour l'application du Deep Learning au diagnostic médical.
- Il illustre comment des technologies modernes peuvent être combinées pour créer des outils d'aide à la décision clinique.
- La prochaine étape naturelle serait une validation clinique rigoureuse avec des radiologues experts.